1. Progressive Electricity Billing

1. Problem Statement

Write a Python program to calculate a household electricity bill using progressive consumption slabs, a fixed charge, and a surcharge when the energy charge exceeds a stated amount.

2. Objectives

• Accept electricity units as input.

• Calculate slab-wise energy charges and a fixed charge.

• Apply a surcharge when the energy charge exceeds the given threshold.

• Display an itemized bill.

3. Assumptions

• Units consumed cannot be negative.

• Slabs are progressive: each rate applies only to units in that slab.

• First 100 units cost ■2/unit; next 100 cost ■3/unit; units above 200 cost ■5/unit.

• Fixed charge is ■50. Add a 5% surcharge to energy charges when energy charges exceed ■500.

4. Algorithm

1. Start.

2. Read units consumed.

3. If units are negative, display an invalid-input message and stop.

4. Calculate charges for each applicable slab.

5. Add the fixed charge.

6. If energy charge is greater than ■500, add a 5% surcharge on energy charge.

7. Display energy charge, fixed charge, surcharge, and total bill.

8. Stop.

5. Flowchart Sketch

See the flowchart diagram on its dedicated page.

e slab

6. Pseudocode

```
START
INPUT units
IF units < 0 THEN
    PRINT "Invalid units"
ELSE
    energy = 0
    IF units <= 100 THEN energy = units * 2
    ELSE IF units <= 200 THEN energy = 100*2 + (units-100)*3
    ELSE energy = 100*2 + 100*3 + (units-200)*5
    fixed = 50
    IF energy > 500 THEN surcharge = energy * 0.05
    ELSE surcharge = 0
    total = energy + fixed + surcharge
    PRINT energy, fixed, surcharge, total
END IF
STOP
```

7. Python Source Code

```
units = int(input("Enter units consumed: "))
if units < 0:
    print("Invalid units")
else:
```

```
if units <= 100:
    energy = units * 2
elif units <= 200:
    energy = 100 * 2 + (units - 100) * 3
else:
    energy = 100 * 2 + 100 * 3 + (units - 200) * 5
fixed = 50
surcharge = energy * 0.05 if energy > 500 else 0
total = energy + fixed + surcharge
print(f"Energy charge: ■{energy:.2f}")
print(f"Fixed charge: ■{fixed:.2f}")
print(f"Surcharge: ■{surcharge:.2f}")
print(f"Total bill: ■{total:.2f}")
```

8. Test Cases

Input / Scenario	Expected Result
50	Energy 100; fixed 50; surcharge 0; total 150
150	Energy 350; fixed 50; surcharge 0; total 400
250	Energy 750; fixed 50; surcharge 37.50; total 837.50
-5	Invalid units

9. Sample Output

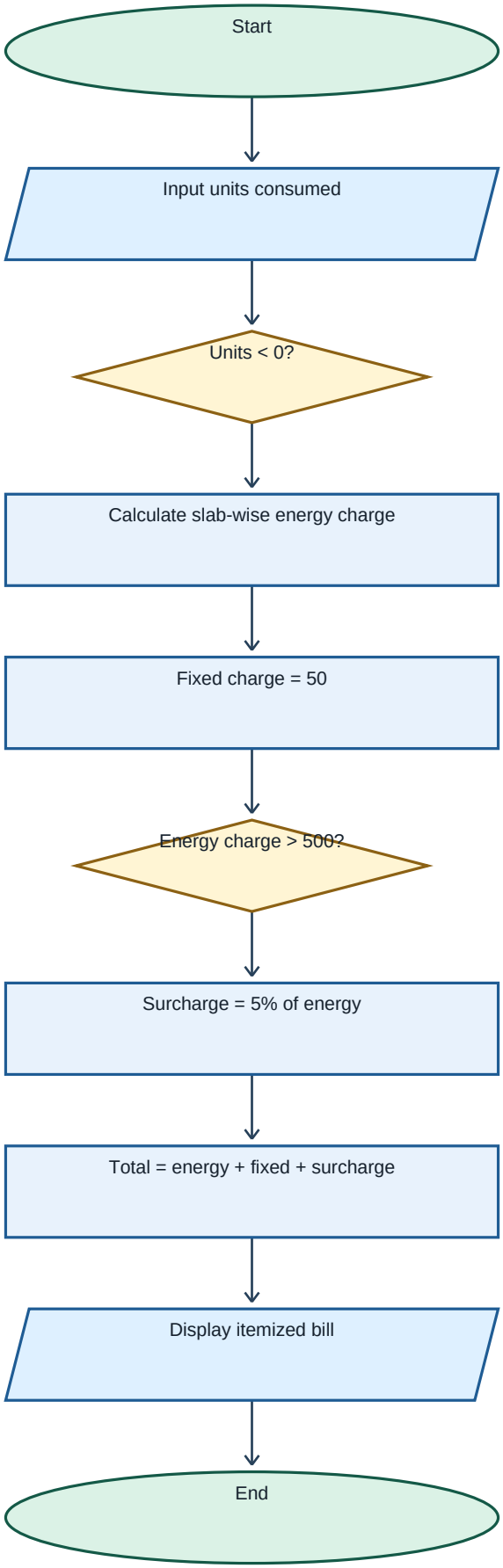
```
Enter units consumed: 250\nEnergy charge: ■750.00\nFixed charge: ■50.00\nSurcharge: ■37.50\nTotal bill: ■837.50
```

10. Conclusion

The program calculates a progressive electricity bill, validates negative input, and reports each charge clearly.

1. Progressive Electricity Billing

Oval = Start/End | Parallelogram = Input/Output | Diamond = Decision | Rectangle = Process



2. Employee Payroll and Deduction Analysis

1. Problem Statement

Create a program that calculates an employee's gross salary from basic pay, allowance, overtime, and incentives, then applies a simple deduction rule and prints a salary statement.

2. Objectives

• Read salary components.

• Calculate gross salary.

• Apply a conditional deduction.

• Print a formatted salary statement.

3. Assumptions

• All input amounts are non-negative.

• Allowance is 20% of basic pay.

• Overtime pay is entered as a total amount.

• Incentive is entered as a total amount.

• Deduction is 10% of gross salary when gross salary is above ■30,000; otherwise it is 5%.

4. Algorithm

1. Start.

2. Input basic pay, overtime pay, and incentive.

3. Reject negative values.

4. Calculate allowance as 20% of basic pay.

5. Calculate gross salary.

6. Select deduction rate using the threshold.

7. Calculate net salary.

8. Display salary details.

9. Stop.

5. Flowchart Sketch

See the flowchart diagram on its dedicated page.

nd\n ↓

6. Pseudocode

```
START
INPUT basic, overtime, incentive
IF any input < 0 THEN PRINT "Invalid input"
ELSE
    allowance = basic * 0.20
    gross = basic + allowance + overtime + incentive
    IF gross > 30000 THEN rate = 0.10
    ELSE rate = 0.05
    deduction = gross * rate
    net = gross - deduction
    PRINT allowance, gross, deduction, net
END IF
STOP
```

7. Python Source Code

```
basic = float(input("Enter basic pay: ■"))
overtime = float(input("Enter overtime pay: ■"))
incentive = float(input("Enter incentive: ■"))
```

```
if basic < 0 or overtime < 0 or incentive < 0:
    print("Invalid input: amounts cannot be negative")
else:
    allowance = basic * 0.20
    gross = basic + allowance + overtime + incentive
    rate = 0.10 if gross > 30000 else 0.05
    deduction = gross * rate
    net = gross - deduction
    print(f"Allowance: ■{allowance:.2f}")
    print(f"Gross salary: ■{gross:.2f}")
    print(f"Deduction: ■{deduction:.2f}")
    print(f"Net salary: ■{net:.2f}")
```

8. Test Cases

Input / Scenario	Expected Result
20000, 1000, 500	Allowance 4000; gross 25500; deduction 1275; net 24225
25000, 2000, 1000	Allowance 5000; gross 33000; deduction 3300; net 29700
-1, 0, 0	Invalid input

9. Sample Output

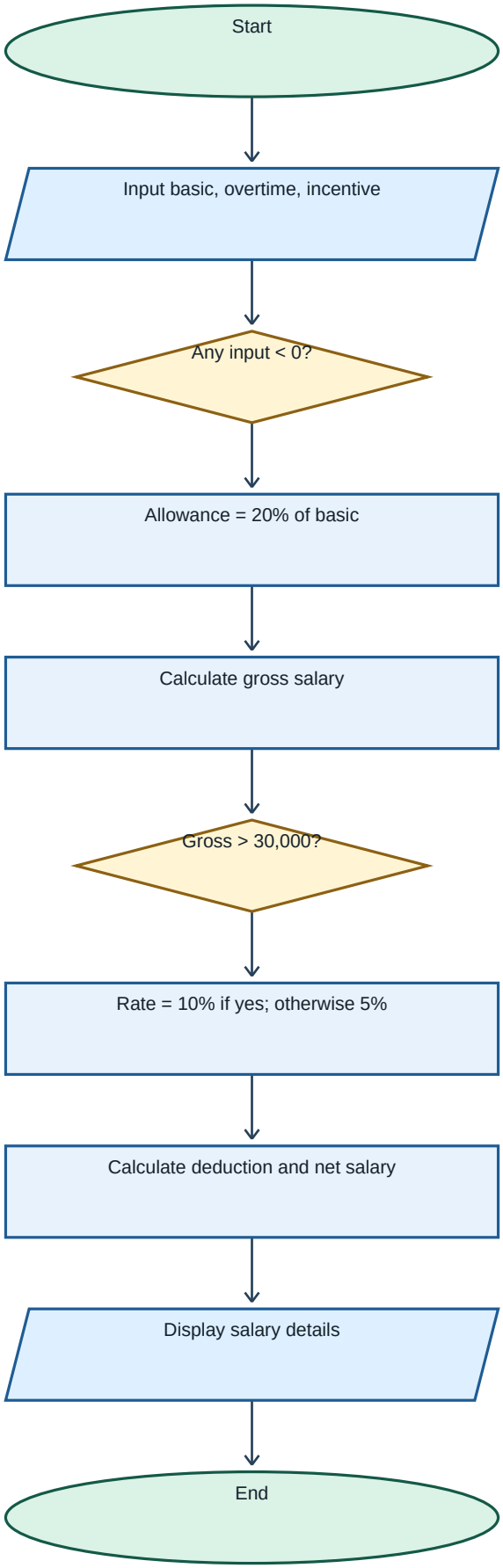
```
Enter basic pay: ■25000\nEnter overtime pay: ■2000\nEnter incentive: ■1000\nAllowance: ■5000.00\nGross salary: ■33000.00
```

10. Conclusion

The program computes gross and net salary by applying the stated allowance and deduction rules.

2. Employee Payroll and Deduction Analysis

Oval = Start/End | Parallelogram = Input/Output | Diamond = Decision | Rectangle = Process



3. Examination Result and Grace-Mark Validation

1. Problem Statement

Write a program that accepts marks in three subjects, checks subject-wise pass marks, applies a grace mark if the student is short by one mark in exactly one subject, and reports the result without exceeding the maximum marks.

2. Objectives

• Read and validate three subject marks.

• Check individual pass criteria.

• Apply a limited grace-mark rule.

• Display total, percentage, and pass/fail result.

3. Assumptions

• Each subject is out of 100 and marks must be from 0 to 100.

• Pass mark is 40 in each subject.

• One grace mark may be added if exactly one subject has 39 and all other subjects have at least 40.

• Grace marks cannot make a subject exceed 100.

4. Algorithm

1. Start.

2. Input marks for three subjects.

3. Validate each mark is between 0 and 100.

4. Count subjects below 40 and identify marks equal to 39.

5. If exactly one subject has 39 and no other subject is below 40, add one grace mark.

6. Check whether every subject is at least 40.

7. Calculate total and percentage.

8. Display adjusted marks and result.

9. Stop.

5. Flowchart Sketch

See the flowchart diagram on its dedicated page.

nYes →

6. Pseudocode

```
START
INPUT m1, m2, m3
IF any mark < 0 OR > 100 THEN PRINT "Invalid marks"
ELSE
    marks = [m1, m2, m3]
    count39 = number of marks equal to 39
    countFail = number of marks below 40
    IF count39 == 1 AND countFail == 1 THEN add 1 to the mark equal to 39
    total = sum(marks)
    percentage = total / 3
    IF every mark >= 40 THEN result = "PASS" ELSE result = "FAIL"
    PRINT marks, total, percentage, result
END IF
STOP
```

7. Python Source Code

```
marks = [int(input(f"Enter marks for subject {i + 1} (0-100): ")) for i in range(3)]
if any(mark < 0 or mark > 100 for mark in marks):
    print("Invalid marks")
else:
```

```
count39 = marks.count(39)
count_fail = sum(1 for mark in marks if mark < 40)
if count39 == 1 and count_fail == 1:
    index = marks.index(39)
    marks[index] += 1
total = sum(marks)
percentage = total / 3
result = "PASS" if all(mark >= 40 for mark in marks) else "FAIL"
print("Adjusted marks:", marks)
print("Total:", total, "/ 300")
print(f"Percentage: {percentage:.2f}%")
print("Result:", result)
```

8. Test Cases

Input / Scenario	Expected Result
39, 60, 70	Adjusted [40, 60, 70]; total 170; 56.67%; PASS
35, 39, 80	No grace; total 154; 51.33%; FAIL
45, 55, 65	Total 165; 55.00%; PASS
101, 40, 40	Invalid marks

9. Sample Output

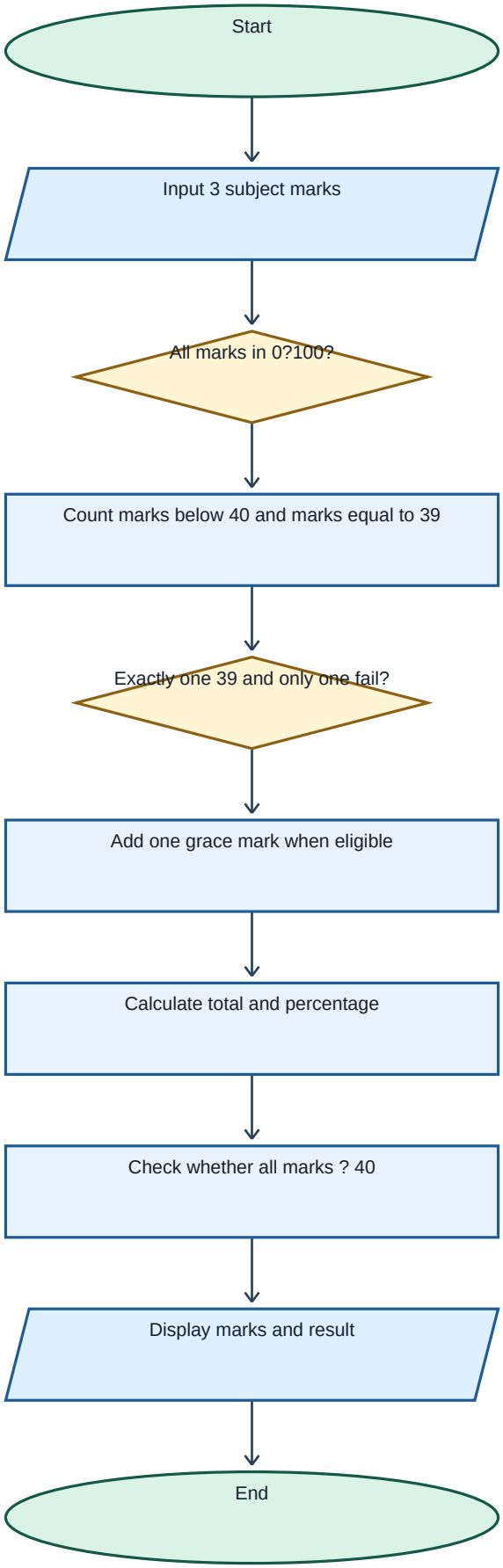
```
Enter marks for subject 1 (0-100): 39\nEnter marks for subject 2 (0-100): 60\nEnter marks for subject 3 (0-100): 70\nAdju
```

10. Conclusion

The program validates marks and applies grace marks only when the stated condition is satisfied.

3. Examination Result and Grace-Mark Validation

Oval = Start/End | Parallelogram = Input/Output | Diamond = Decision | Rectangle = Process



4. Attendance Recovery Computation

1. Problem Statement

Calculate a student's current attendance percentage and determine the minimum number of consecutive classes the student must attend to reach a target attendance percentage.

2. Objectives

• Calculate current attendance percentage.

• Validate class counts.

• Compute the minimum future classes needed.

• Show the result clearly.

3. Assumptions

• Classes held must be greater than zero.

• Classes attended must be between zero and classes held.

• The target attendance is 75%.

• The student attends every future class.

4. Algorithm

1. Start.

2. Input classes held and classes attended.

3. Validate the counts.

4. Calculate current attendance percentage.

5. If attendance already meets 75%, display zero required classes.

6. Otherwise increase the future class count until the attendance ratio reaches 75%.

7. Display current percentage and required classes.

8. Stop.

5. Flowchart Sketch

See the flowchart diagram on its dedicated page.

Yes → (

6. Pseudocode

```
START
INPUT held, attended
IF held <= 0 OR attended < 0 OR attended > held THEN PRINT "Invalid input"
ELSE
    current = attended / held * 100
    future = 0
    WHILE (attended + future) / (held + future) < 0.75
        future = future + 1
    PRINT current, future
END IF
STOP
```

7. Python Source Code

```
held = int(input("Enter total classes held: "))
attended = int(input("Enter classes attended: "))
if held <= 0 or attended < 0 or attended > held:
    print("Invalid class counts")
else:
    current = attended / held * 100
    future = 0
    while (attended + future) / (held + future) < 0.75:
        future += 1
```

```
print(f"Current attendance: {current:.2f}%")
print("Consecutive classes required:", future)
```

8. Test Cases

Input / Scenario	Expected Result
60, 42	Current 70.00%; required 12
80, 64	Current 80.00%; required 0
20, 10	Current 50.00%; required 20
0, 0	Invalid class counts

9. Sample Output

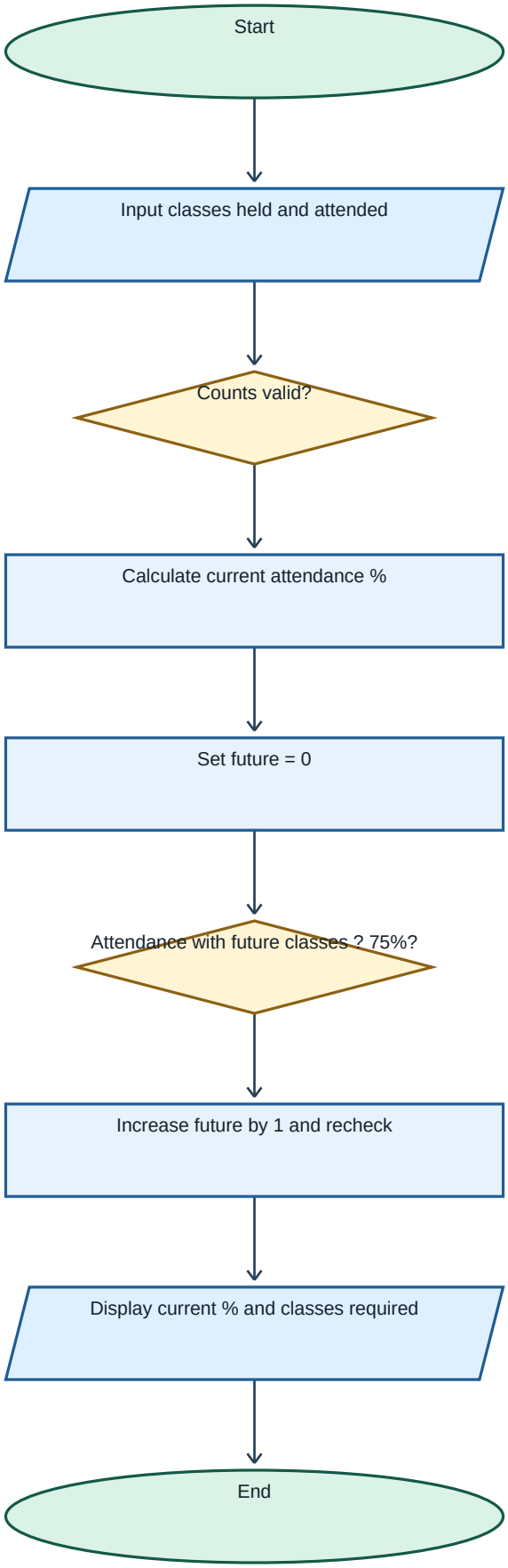
```
Enter total classes held: 60\nEnter classes attended: 42\nCurrent attendance: 70.00%\nConsecutive classes required: 12
```

10. Conclusion

The program calculates current attendance and uses a loop to find the minimum future classes needed to meet the target.

4. Attendance Recovery Computation

Oval = Start/End | Parallelogram = Input/Output | Diamond = Decision | Rectangle = Process



5. ATM Transaction Sequence Simulation

1. Problem Statement

Simulate a sequence of ATM deposits and withdrawals, maintaining the account balance and rejecting invalid transactions or withdrawals that exceed the available balance.

2. Objectives

• Process a list of transactions.

• Update balance after valid transactions.

• Reject invalid transaction amounts and overdraw attempts.

• Summarize accepted and rejected transactions.

3. Assumptions

• Opening balance is ■1,000.

• Transactions are entered as type and amount pairs.

• Only DEPOSIT and WITHDRAW are valid transaction types.

• Amounts must be positive.

• No overdraft is permitted.

4. Algorithm

1. Start.

2. Set the opening balance to ■1,000.

3. Input the number of transactions.

4. For each transaction, read type and amount.

5. Reject invalid types or non-positive amounts.

6. For a withdrawal, reject it if amount exceeds balance.

7. Otherwise update the balance and accepted count.

8. Display final balance and accepted/rejected counts.

9. Stop.

5. Flowchart Sketch

See the flowchart diagram on its dedicated page.

d type

6. Pseudocode

```
START
balance = 1000
accepted = 0; rejected = 0
INPUT n
REPEAT n times
    INPUT type, amount
    IF amount <= 0 OR type not in [DEPOSIT, WITHDRAW] THEN rejected += 1
    ELSE IF type == WITHDRAW AND amount > balance THEN rejected += 1
    ELSE IF type == DEPOSIT THEN balance += amount; accepted += 1
    ELSE balance -= amount; accepted += 1
PRINT balance, accepted, rejected
STOP
```

7. Python Source Code

```
balance = 1000.0
accepted = 0
rejected = 0
n = int(input("Number of transactions: "))
```

```
for i in range(n):
    kind = input("Transaction type (DEPOSIT/WITHDRAW): ").strip().upper()
    amount = float(input("Amount: ■"))
    if amount <= 0 or kind not in ("DEPOSIT", "WITHDRAW"):
        print("Transaction rejected")
        rejected += 1
    elif kind == "WITHDRAW" and amount > balance:
        print("Transaction rejected: insufficient balance")
        rejected += 1
    else:
        if kind == "DEPOSIT":
            balance += amount
        else:
            balance -= amount
        print("Transaction accepted")
        accepted += 1
print(f"Final balance: ■{balance:.2f}")
print("Accepted transactions:", accepted)
print("Rejected transactions:", rejected)
```

8. Test Cases

Input / Scenario	Expected Result
DEPOSIT 500; WITHDRAW 200	Final balance 1300; accepted 2; rejected 0
WITHDRAW 1500	Rejected; balance remains 1000
DEPOSIT -10	Rejected; balance remains 1000
TRANSFER 100	Rejected invalid type

9. Sample Output

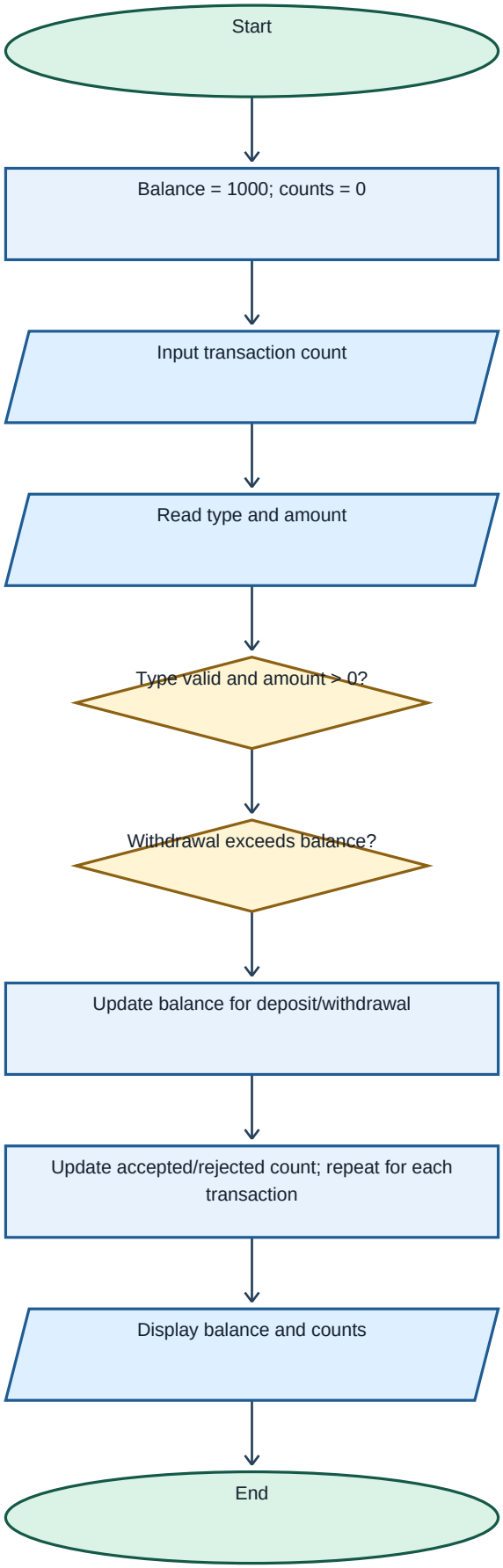
```
Number of transactions: 2\nTransaction type (DEPOSIT/WITHDRAW): DEPOSIT\nAmount: ■500\nTransaction accepted\nTransaction
```

10. Conclusion

The simulation maintains a running balance and enforces basic transaction validation without allowing overdrafts.

5. ATM Transaction Sequence Simulation

Oval = Start/End | Parallelogram = Input/Output | Diamond = Decision | Rectangle = Process



6. Warehouse Inventory Reconciliation

1. Problem Statement

Compare opening stock, received quantities, and dispatched quantities for multiple products; calculate expected closing stock, compare it with recorded closing stock, and identify discrepancies.

2. Objectives

• Store product data in lists.

• Calculate expected closing stock.

• Compare expected and recorded stock.

• Report discrepancies and replenishment needs.

3. Assumptions

• Opening, received, dispatched, and recorded closing quantities are non-negative integers.

• Expected closing stock = opening + received – dispatched.

• If expected stock is below 10 units, report the quantity needed to reach 10.

4. Algorithm

1. Start.

2. Input the number of products.

3. For each product, input its name and stock quantities.

4. Validate quantities and ensure dispatched stock does not exceed available stock.

5. Calculate expected closing stock.

6. Compare expected stock with recorded stock.

7. Calculate replenishment quantity if expected stock is below 10.

8. Display a report for each product.

9. Stop.

5. Flowchart Sketch

See the flowchart diagram on its dedicated page.

spatch

6. Pseudocode

```
START
INPUT n
FOR each product
    INPUT name, opening, received, dispatched, recorded
    IF any quantity < 0 OR dispatched > opening + received THEN PRINT "Invalid record"
    ELSE
        expected = opening + received - dispatched
        difference = recorded - expected
        replenish = max(0, 10 - expected)
        PRINT expected, difference, replenish
STOP
```

7. Python Source Code

```
n = int(input("Number of products: "))
for i in range(n):
    name = input("Product name: ")
    opening = int(input("Opening stock: "))
    received = int(input("Received quantity: "))
    dispatched = int(input("Dispatched quantity: "))
    recorded = int(input("Recorded closing stock: "))
    if min(opening, received, dispatched, recorded) < 0 or dispatched > opening + received:
        print("Invalid stock record")
```

```
    continue
expected = opening + received - dispatched
difference = recorded - expected
replenish = max(0, 10 - expected)
print("Product:", name)
print("Expected closing stock:", expected)
print("Recorded minus expected:", difference)
print("Replenishment needed:", replenish)
print("Discrepancy:", "No" if difference == 0 else "Yes")
```

8. Test Cases

Input / Scenario	Expected Result
Pen: opening 20, received 5, dispatched 10, recorded 15	Expected 15; difference 0; replenish 0; no discrepancy
Book: 5, 0, 2, 4	Expected 3; difference 1; replenish 7; discrepancy yes
Item: 3, 2, 8, 0	Invalid stock record

9. Sample Output

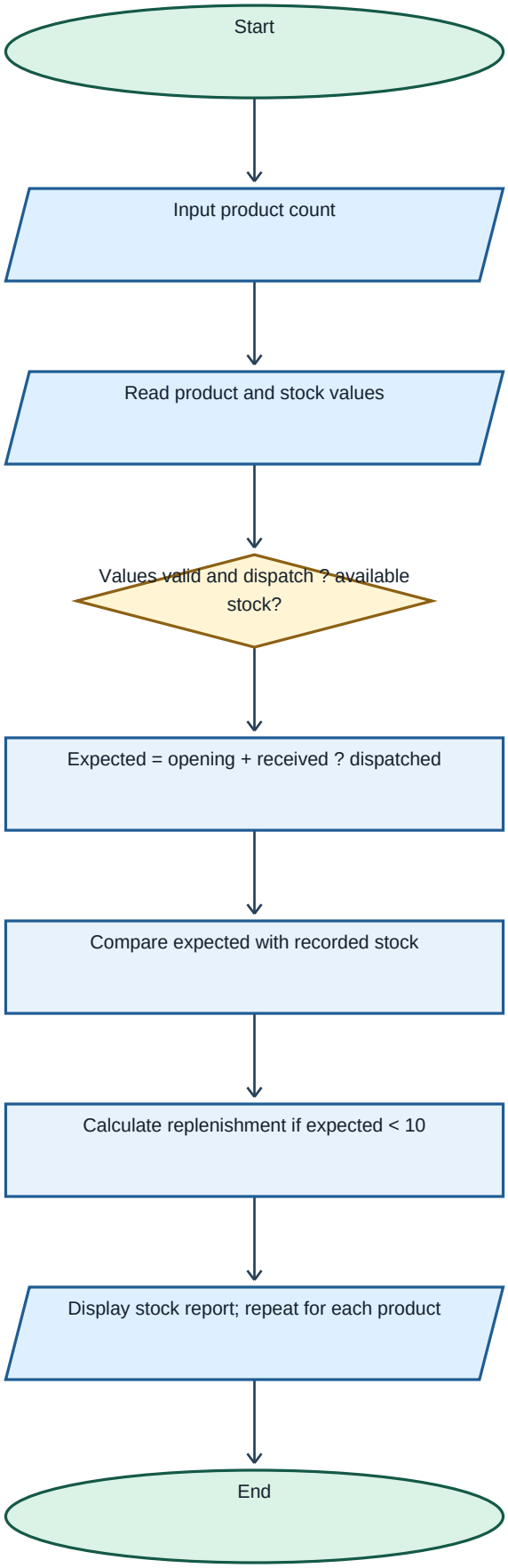
```
Number of products: 1\nProduct name: Pen\nOpening stock: 20\nReceived quantity: 5\nDispatched quantity: 10\nRecorded clos
```

10. Conclusion

The program reconciles stock records, identifies mismatches, and calculates replenishment quantities.

6. Warehouse Inventory Reconciliation

Oval = Start/End | Parallelogram = Input/Output | Diamond = Decision | Rectangle = Process



7. Student Marks Matrix Analysis

1. Problem Statement

Store marks for multiple students and subjects in a nested list, calculate each student's total and average, and calculate each subject's average to identify the strongest student and weakest-average subject.

2. Objectives

• Represent marks using a nested list.

• Calculate row-wise student totals and averages.

• Calculate column-wise subject averages.

• Identify the top student and lowest-average subject.

3. Assumptions

• There are three students and three subjects in this example.

• Each mark is an integer from 0 to 100.

• Student names and subject names are supplied in the program.

4. Algorithm

1. Start.

2. Read marks for each student into a nested list.

3. Validate that every mark is between 0 and 100.

4. Calculate each student's total and average.

5. Calculate average marks for each subject.

6. Find the student with the highest total.

7. Find the subject with the lowest average.

8. Display all results.

9. Stop.

5. Flowchart Sketch

See the flowchart diagram on its dedicated page.

\nYes –

6. Pseudocode

```
START
students = [Asha, Ravi, Meena]
subjects = [Python, Maths, Web]
INPUT marks as 3 x 3 nested list
IF any mark < 0 OR > 100 THEN PRINT "Invalid marks"
ELSE
    FOR each student row
        total = sum(row); average = total / 3
    FOR each subject column
        subject_average = sum(column marks) / 3
    top_student = student with maximum total
    weakest_subject = subject with minimum average
    PRINT report
STOP
```

7. Python Source Code

```
students = ["Asha", "Ravi", "Meena"]
subjects = ["Python", "Maths", "Web"]
marks = []
for student in students:
    row = []
    for subject in subjects:
```

```
        mark = int(input(f"{student} - {subject} marks: "))
        row.append(mark)
    marks.append(row)
if any(mark < 0 or mark > 100 for row in marks for mark in row):
    print("Invalid marks: enter values from 0 to 100")
else:
    totals = []
    for i, row in enumerate(marks):
        total = sum(row)
        totals.append(total)
        print(f"{students[i]}: total={total}, average={total / len(subjects):.2f}")
    subject_averages = []
    for j, subject in enumerate(subjects):
        average = sum(marks[i][j] for i in range(len(students))) / len(students)
        subject_averages.append(average)
        print(f"{subject} average: {average:.2f}")
    print("Top student:", students[totals.index(max(totals))])
    print("Lowest-average subject:", subjects[subject_averages.index(min(subject_averages))])
```

8. Test Cases

Input / Scenario	Expected Result
Rows [80,70,90]; [60,85,75]; [90,90,95]	Totals 240, 220, 275; top student Meena
All marks 0	All averages 0; first student and first subject returned in ties
101 in any subject	Invalid marks message

9. Sample Output

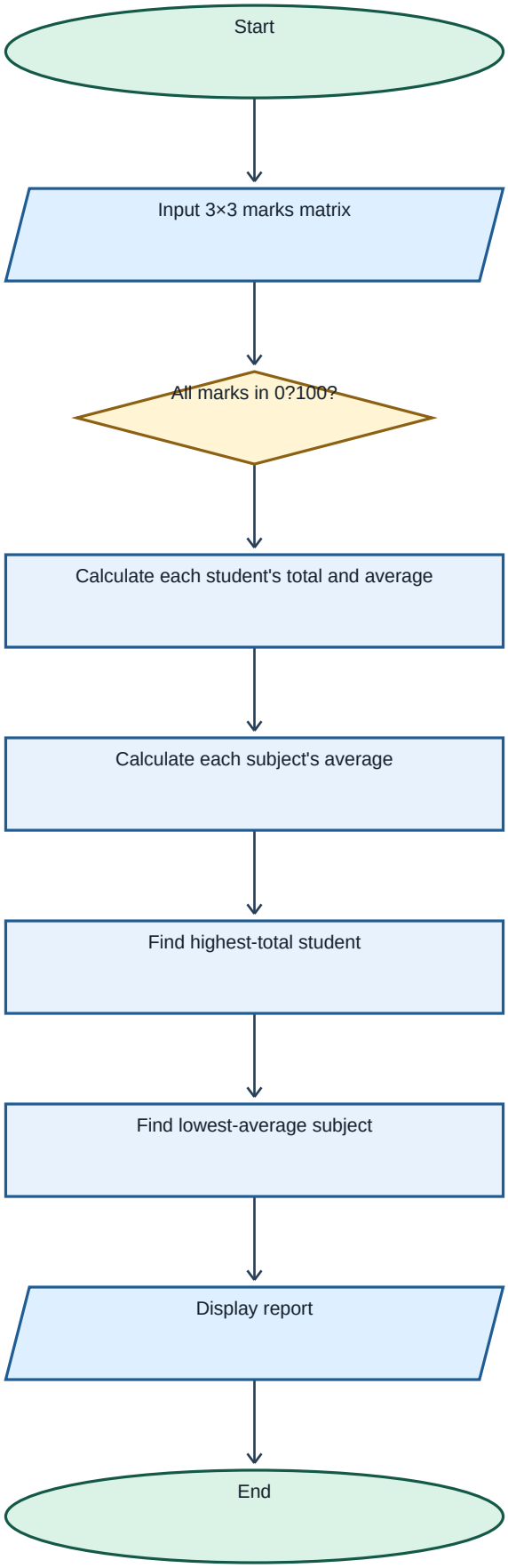
```
Asha - Python marks: 80\nAsha - Maths marks: 70\nAsha - Web marks: 90\nRavi - Python marks: 60\nRavi - Maths marks: 85\nR
```

10. Conclusion

The nested-list solution calculates student and subject statistics and identifies top and weak performance areas.

7. Student Marks Matrix Analysis

Oval = Start/End | Parallelogram = Input/Output | Diamond = Decision | Rectangle = Process



8. Smart Canteen Order and Stock Management

1. Problem Statement

Develop a simple canteen program that accepts an order for menu items, calculates the bill, updates stock, rejects orders exceeding available quantities, and displays remaining stock.

2. Objectives

• Store menu prices and stock in lists.

• Accept item and quantity selections.

• Calculate order totals.

• Update inventory and reject unavailable quantities.

3. Assumptions

• The menu contains Idli (■30, stock 20), Dosa (■50, stock 15), and Sandwich (■60, stock 10).

• Menu item numbers are 1 to 3.

• Quantities must be positive integers.

• A valid order updates stock immediately.

4. Algorithm

1. Start.

2. Create menu, price, and stock lists.

3. Display the menu.

4. Input item number and quantity.

5. Validate item number and quantity.

6. If requested quantity exceeds stock, reject the order.

7. Otherwise calculate cost and reduce stock.

8. Display bill and remaining stock.

9. Stop.

5. Flowchart Sketch

See the flowchart diagram on its dedicated page.

\nVal

6. Pseudocode

```
START
menu = [Idli, Dosa, Sandwich]
prices = [30, 50, 60]
stock = [20, 15, 10]
INPUT item_number, quantity
IF item_number not in 1..3 OR quantity <= 0 THEN PRINT "Invalid order"
ELSE
    index = item_number - 1
    IF quantity > stock[index] THEN PRINT "Insufficient stock"
    ELSE
        total = prices[index] * quantity
        stock[index] = stock[index] - quantity
        PRINT total, stock
STOP
```

7. Python Source Code

```
menu = ["Idli", "Dosa", "Sandwich"]
prices = [30, 50, 60]
stock = [20, 15, 10]
print("MENU")
```

```
for i in range(len(menu)):
    print(f"{i + 1}. {menu[i]} - ■{prices[i]} (stock: {stock[i]})")
item = int(input("Choose item number: "))
quantity = int(input("Enter quantity: "))
if item < 1 or item > len(menu) or quantity <= 0:
    print("Invalid order")
else:
    index = item - 1
    if quantity > stock[index]:
        print("Insufficient stock")
    else:
        total = prices[index] * quantity
        stock[index] -= quantity
        print("Item:", menu[index])
        print("Bill amount: ■", total)
        print("Remaining stock:", stock[index])
```

8. Test Cases

Input / Scenario	Expected Result
Item 1, quantity 2	Idli; bill ■60; remaining stock 18
Item 2, quantity 20	Insufficient stock
Item 4, quantity 1	Invalid order
Item 3, quantity 0	Invalid order

9. Sample Output

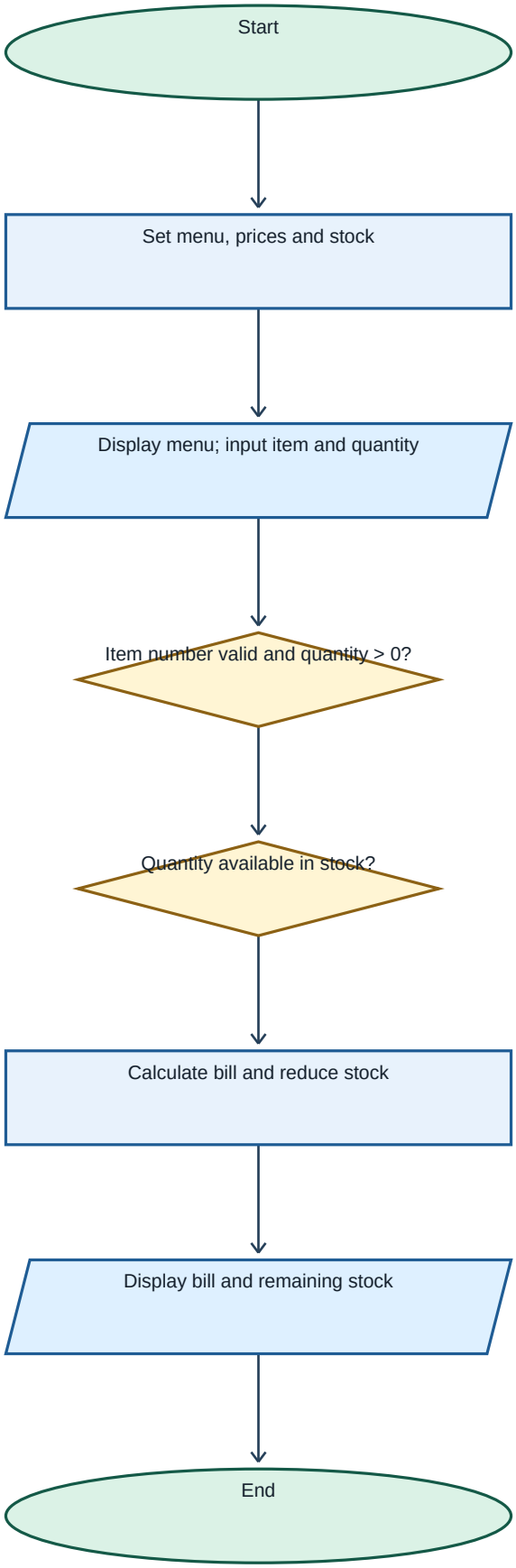
```
MENU\n1. Idli - ■30 (stock: 20)\n2. Dosa - ■50 (stock: 15)\n3. Sandwich - ■60 (stock: 10)\nChoose item number: 1\nEnter q
```

10. Conclusion

The program handles a basic canteen order, calculates its bill, and keeps stock quantities consistent.

8. Smart Canteen Order and Stock Management

Oval = Start/End | Parallelogram = Input/Output | Diamond = Decision | Rectangle = Process



9. Nested Loop Examination Seating Generator

1. Problem Statement

Generate a classroom seating layout using nested loops. Print seat identifiers by row and column, and mark selected positions as reserved.

2. Objectives

• Use nested loops to generate a grid.

• Create row-column seat identifiers.

• Mark reserved seats.

• Display a clear seating arrangement.

3. Assumptions

• The classroom has 4 rows and 5 columns.

• Seats R1C3 and R3C2 are reserved.

• All other seats are available.

• Row and column numbering begins at 1.

4. Algorithm

1. Start.

2. Set the number of rows and columns.

3. Store reserved seat identifiers in a list.

4. Use an outer loop for rows and an inner loop for columns.

5. Build each seat identifier.

6. Print RESERVED for reserved seats and the identifier for available seats.

7. Stop.

5. Flowchart Sketch

See the flowchart diagram on its dedicated page.

↓ \n

6. Pseudocode

```
START
rows = 4; columns = 5
reserved = [R1C3, R3C2]
FOR r = 1 TO rows
    FOR c = 1 TO columns
        seat = "R" + r + "C" + c
        IF seat in reserved THEN PRINT "RESERVED"
        ELSE PRINT seat
    PRINT newline
STOP
```

7. Python Source Code

```
rows = 4
columns = 5
reserved = ["R1C3", "R3C2"]
for r in range(1, rows + 1):
    row_seats = []
    for c in range(1, columns + 1):
        seat = f"R{r}C{c}"
        row_seats.append("RESERVED" if seat in reserved else seat)
    print(" | ".join(row_seats))
```

8. Test Cases

Input / Scenario	Expected Result
Default layout	20 seats displayed; R1C3 and R3C2 marked RESERVED
Change rows to 2	Two rows displayed
Reserved list empty	All seats display identifiers

9. Sample Output

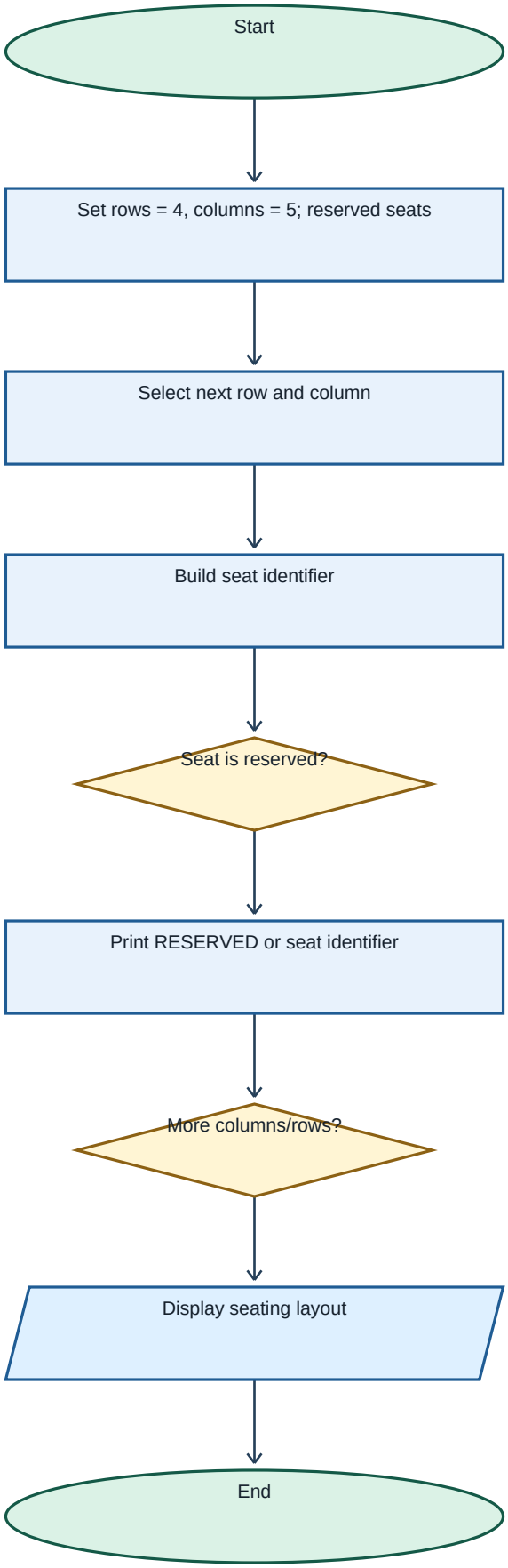
```
R1C1 | R1C2 | RESERVED | R1C4 | R1C5\nR2C1 | R2C2 | R2C3 | R2C4 | R2C5\nR3C1 | RESERVED | R3C3 | R3C4 | R3C5\nR4C1 | R4C2
```

10. Conclusion

Nested loops efficiently generate a seating grid and apply reserved-seat rules.

9. Nested Loop Examination Seating Generator

Oval = Start/End | Parallelogram = Input/Output | Diamond = Decision | Rectangle = Process



10. Tuple-Based Product Catalogue Audit

1. Problem Statement

Store product details in tuples containing product ID, name, price, and stock. Identify invalid records, calculate inventory value for valid records, and flag products that need replenishment.

2. Objectives

• Represent product records as tuples.

• Validate price and stock values.

• Calculate inventory value.

• Identify items below the stock threshold.

3. Assumptions

• Each record is (product_id, name, price, stock).

• Price and stock must be non-negative.

• A product needs replenishment when stock is below 5 units.

• The sample catalogue is stored in the program.

4. Algorithm

1. Start.

2. Create a list of product tuples.

3. For each tuple, unpack its fields.

4. If price or stock is negative, mark the record invalid.

5. Otherwise calculate price multiplied by stock.

6. If stock is below 5, flag replenishment.

7. Display valid values, invalid records, and total inventory value.

8. Stop.

5. Flowchart Sketch

See the flowchart diagram on its dedicated page.

?\n ↓

6. Pseudocode

```
START
products = [(P101, Pen, 10, 20), (P102, Book, 80, 3), (P103, Bag, 500, -1)]
total_value = 0
FOR each (id, name, price, stock) in products
    IF price < 0 OR stock < 0 THEN PRINT invalid record
    ELSE
        value = price * stock
        total_value += value
        IF stock < 5 THEN PRINT replenishment needed
        PRINT value
PRINT total_value
STOP
```

7. Python Source Code

```
products = [("P101", "Pen", 10, 20),
            ("P102", "Book", 80, 3),
            ("P103", "Bag", 500, -1)]
total_value = 0
for product_id, name, price, stock in products:
    if price < 0 or stock < 0:
        print(f"{product_id} - {name}: Invalid record")
        continue
```

```
    value = price * stock
    total_value += value
    print(f"{product_id} - {name}: inventory value ■{value}")
    if stock < 5:
        print("  Replenishment required")
print(f"Total valid inventory value: ■{total_value}")
```

8. Test Cases

Input / Scenario	Expected Result
P101 Pen 10 20	Value ■200; no replenishment
P102 Book 80 3	Value ■240; replenishment required
P103 Bag 500 -1	Invalid record
Full sample	Total valid inventory value ■440

9. Sample Output

```
P101 - Pen: inventory value ■200\nP102 - Book: inventory value ■240\n  Replenishment required\nP103 - Bag: Invalid record
```

10. Conclusion

The tuple-based audit validates product records, computes stock value, and identifies low-stock items.

10. Tuple-Based Product Catalogue Audit

Oval = Start/End | Parallelogram = Input/Output | Diamond = Decision | Rectangle = Process

